

1. Use ArrayList for Ordered Dynamic Storage (Interview Notes)

What is ArrayList?

ArrayList is a **resizable array** implementation of the List interface.

It stores elements in **insertion order**, allows **duplicates**, and automatically grows when needed.

```
ArrayList fruits = new ArrayList<>();  
  
fruits.add("Apple");  
fruits.add("Banana");  
fruits.add("Apple");
```

Output

```
Apple  
Banana  
Apple
```

Duplicates are allowed.

Why Use ArrayList?

Use ArrayList when you need:

- Dynamic size
 - Fast random access (index-based)
 - Ordered elements
 - Frequent reading of data
-

Important Interview Points

- ArrayList implements the **List** interface.
- Maintains **insertion order**.

- Allows **duplicate** and **null** elements.
 - Supports **index-based access** (get(index)).
 - Internally uses a **dynamic array**.
 - Automatically grows when capacity becomes full.
 - Best choice when **read operations are more frequent than insertions/deletions**.
 - Not synchronized (use Vector or Collections.synchronizedList() if thread safety is required).
-

Tricky Interview Questions

Q1. Does ArrayList allow duplicates?

Answer: Yes.

Q2. Does ArrayList maintain insertion order?

Answer: Yes.

Q3. Can ArrayList store null values?

Answer: Yes, multiple null values are allowed.

Q4. Is ArrayList synchronized?

Answer: No.

It is **not thread-safe** by default.

Q5. Why is ArrayList preferred over an array?

Answer:

Because it provides:

- Dynamic resizing
 - Rich built-in methods
 - Better flexibility
-

Interview Summary

- **ArrayList** is a **dynamic, ordered implementation of the List interface**.
- Maintains insertion order and allows duplicates.
- Uses a **dynamic array** internally.
- Provides fast index-based access.
- Best for applications with **frequent reads and dynamic data size**.

Interview One-Liner:

"ArrayList is a **resizable array** that maintains insertion order, allows duplicates, and provides fast index-based access."

2. Understand ArrayList Internal Resizing Concept (Interview Notes)

How Does ArrayList Grow?

Internally, ArrayList uses a **regular array**.

When the internal array becomes full:

1. A **larger array** is created.
2. Existing elements are copied into the new array.
3. The old array becomes eligible for Garbage Collection.
4. The new element is inserted.

This process happens automatically.

Example

```
ArrayList list = new ArrayList<>();

for(int i = 1; i <= 20; i++) {
    list.add(i);
}
```

Even though you didn't specify the size, ArrayList automatically increases its capacity.

Internal Working

Initial Capacity

[10]

↓

Add 11th Element

↓

Create Larger Array

↓

Copy Old Elements

↓

Insert New Element

Java generally increases the capacity by **about 50%** ($\text{oldCapacity} + \text{oldCapacity}/2$).

Important Interview Points

- ArrayList stores data in an internal **array**.
- Capacity and **size** are different:
 - **Size** = Number of stored elements.
 - **Capacity** = Total space available before resizing.
- Resizing occurs **automatically** when capacity is exceeded.
- Resizing involves creating a new array and copying all existing elements.
- Frequent resizing is expensive, so if the approximate number of elements is known, specify the **initial capacity**.

Example:

```
ArrayList list = new ArrayList<>(1000);
```

Tricky Interview Questions

Q1. Does ArrayList resize after every insertion?

Answer: No.

Only when the internal capacity becomes full.

Q2. What happens internally during resizing?

Answer:

- New larger array created
 - Elements copied
 - Old array discarded
-

Q3. Difference between size and capacity?

Answer:

- Size = Stored elements
 - Capacity = Allocated space
-

Q4. Why specify initial capacity?

Answer:

To reduce resizing operations and improve performance.

Q5. Is resizing free?

Answer: No.

Copying all elements makes resizing an **O(n)** operation.

Interview Summary

- ArrayList internally uses a **dynamic array**.
- When full, it creates a **larger array**, copies existing elements, and continues storing data.
- **Capacity** is the allocated storage; **size** is the number of elements currently stored.

- Specifying an initial capacity reduces expensive resize operations.
- Automatic resizing provides flexibility but has a temporary performance cost.

Interview One-Liner:

"ArrayList automatically resizes by creating a larger internal array and copying existing elements when its capacity is exceeded."

3. Analyze ArrayList add(), get(), remove() Time Complexity (Interview Notes)

Common Time Complexities

Operation	Time Complexity
get(index)	$O(1)$
add(element) (end)	$O(1)$ (Amortized)
add(index)	$O(n)$
remove(index)	$O(n)$
contains()	$O(n)$

Why?

1. get(index) → $O(1)$

```
list.get(5);
```

Because ArrayList uses an array, the JVM can directly calculate the memory location.

No searching required.

2. add(element) → $O(1)$ Amortized

```
list.add(100);
```

Normally, the element is placed at the end.

If resizing is needed:

- New array created
- All elements copied

That insertion becomes $O(n)$.

Overall average remains **$O(1)$ Amortized**.

3. `add(index) → $O(n)$`

```
list.add(2, "Java");
```

Elements after index 2 must shift one position to the right.

4. `remove(index) → $O(n)$`

```
list.remove(2);
```

Elements after index 2 shift one position to the left.

Important Interview Points

- `get(index)` is **$O(1)$** due to direct array indexing.
 - `add(element)` at the end is **$O(1)$ amortized**; resizing occasionally makes it **$O(n)$** .
 - `add(index)` and `remove(index)` are **$O(n)$** because elements must be shifted.
 - Accessing elements is much faster than inserting/removing in the middle.
 - `ArrayList` is ideal for **read-heavy** applications but less efficient for frequent middle insertions/deletions.
-

Tricky Interview Questions

Q1. Why is `get()` $O(1)$?

Answer:

Because arrays support direct index access.

Q2. Why is `add()` called $O(1)$ Amortized?

Answer:

Most insertions are $O(1)$; only occasional resizing makes a single insertion $O(n)$.

Q3. Why is `remove(index)` $O(n)$?

Answer:

Remaining elements must shift left to fill the gap.

Q4. Which is faster for inserting in the middle: `ArrayList` or `LinkedList`?

Answer:

LinkedList, because it doesn't shift elements (assuming you've already reached the insertion position).

Q5. Which operation is the fastest in `ArrayList`?

Answer:

`get(index)` — $O(1)$.

Interview Summary ★

- `get(index)` → **$O(1)$** due to direct array access.
- `add(element)` at the end → **$O(1)$ amortized**; resizing occasionally makes it **$O(n)$** .
- `add(index)` and `remove(index)` → **$O(n)$** because elements must be shifted.
- `ArrayList` excels at **fast random access** but is less efficient for frequent insertions or deletions in the middle.
- Understanding these time complexities is essential for choosing the right collection in interviews.

Interview One-Liner:

"ArrayList provides $O(1)$ random access and $O(1)$ amortized insertion at the end, but insertions and deletions in the middle are $O(n)$ because elements must be shifted."